Entrega 2 — Predicción de Churn en E-commerce (Olist)

Modelos, MLflow en EC2 y Tablero

David Alan Delgado Zarco
Octavio Esquivel Alvarez del Castillo
John Williams Morrill
Jason Rosher Morrill

March 10, 2026

Abstract

Este documento presenta la implementación de un modelo de predicción de churn (abandono) para clientes del dataset Olist, siguiendo la definición operacional de la Entrega 1 con ventana $W = 90$ días. Para evitar *leakage* temporal, el label se define observando compras en una ventana futura posterior a una fecha de *snapshot*. Se construyen features a nivel `customer_unique_id` (RFM y experiencia), se aplica una partición temporal train/test y se comparan modelos (Regresión Logística vs. Random Forest), evaluando especialmente la clase minoritaria (no-churn/recompra). Los experimentos se registran y comparan en MLflow desplegado en AWS EC2, y se construye un tablero funcional (Streamlit) que muestra predicciones, umbral operativo y visualizaciones macro.

1 Cambios vs. Entrega 1

- Se corrigió la definición del target: se eliminó el criterio trivial `churned = Recency > W` y se implementó un esquema *snapshot + ventana futura* $(t, t + W]$.
- Se evitó *leakage* temporal: features calculadas solo con historial previo al snapshot y partición temporal (no aleatoria).
- Se ajustó la población objetivo usando un *lookback* para enfocarse en clientes activos (evita incluir clientes históricamente inactivos).
- Se amplió el soporte experimental: ejecución y comparación de runs en MLflow en una instancia AWS EC2 (con evidencias de IP/usuario).
- Se implementó un tablero funcional que consume el modelo ganador y muestra predicción, umbral y vistas macro.

2 Datos y construcción del label

Fuentes de datos

Se utilizaron los archivos del dataset Olist: `orders`, `customers`, `payments`, `reviews` e `items`. Para evitar duplicación por relaciones 1:N, se agregaron pagos, reviews e items por `order_id` antes de construir el dataset a nivel cliente.

Construcción del label (churn)

Se definió una fecha de *snapshot* y una ventana futura $W = 90$ días. El churn se define como:

$$churn = 1 \Leftrightarrow \text{el cliente no realiza compras en } (snapshot, snapshot + W]$$

y $churn = 0$ en caso contrario (recompra). Para evitar sesgos por clientes históricamente inactivos, se filtra la población a clientes con compra reciente dentro de un *lookback* previo al snapshot.

Evidencia de integridad del dataset

En el preprocesamiento a nivel orden se verificó que no hubiera duplicación de `order_id` (duplicados = 0), evitando inflación artificial de montos o conteos.

3 Features y partición temporal

Features

Se construyeron features a nivel `customer_unique_id`:

- **RFM**: Recency, Frequency, Monetary.
- **Experiencia/logística**: promedio de review score, días de entrega, retraso vs. fecha estimada, número de items, suma de precio y flete.
- **Contexto**: estado del cliente (`customer_state`).

Partición temporal

Se realizó un split temporal basado en la última compra del cliente (previa al snapshot), con un percentil (p.ej. 80%) para separar train/test. Esto evita *leakage* y refleja un escenario realista de generalización temporal.

4 Modelos y evaluación

Baseline por desbalance

El dataset presenta alto desbalance (baja recompra). Por ello, se reportó un baseline trivial que predice churn para todos: logra accuracy alta pero F1 para no-churn es 0. Esto justifica el uso de métricas enfocadas a la clase minoritaria.

Modelos

Se compararon:

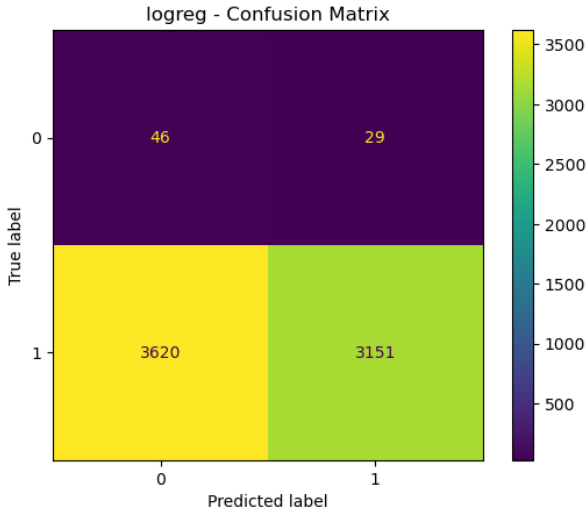
- **Regresión logística** (con `class_weight=balanced`), variando C y el umbral de decisión.
- **Random Forest** (con `class_weight=balanced_subsample`), con variantes de `max_depth` y `min_samples_leaf`.

Métricas

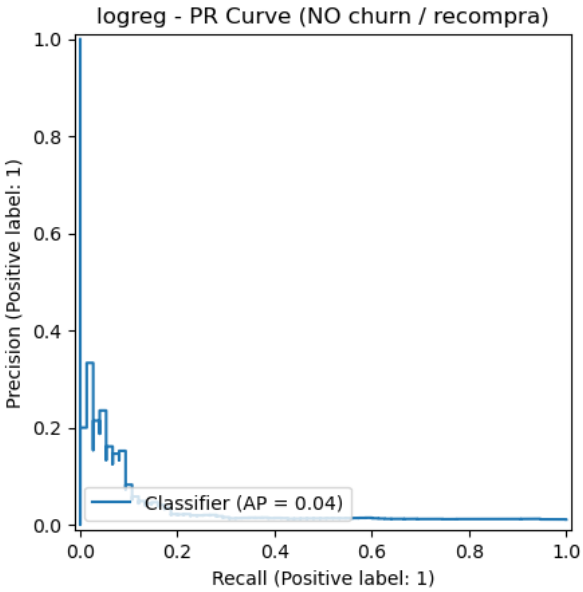
Además de accuracy, se reportaron métricas relevantes para no-churn (recompra): `pr_auc_nochurn_0` y `f1_nochurn_0`, junto con matrices de confusión y curvas Precision–Recall para la clase no-churn.

Selección de umbral

Se optimizó el umbral para maximizar `f1_nochurn_0`. En los experimentos se obtuvo un umbral operativo de 0.12.
Run ganador (MLflow). Se seleccionó el run `logreg_C0.1_thr012` por obtener el mejor valor de `pr_auc_nochurn_0` (y un alto `f1_nochurn_0`) en la comparación de runs.



(a) Matriz de confusión (LogReg).



(b) Curva PR para no-churn (LogReg).

Figure 1: Evaluación del modelo.

5 Soportes en MLflow (AWS EC2)

Despliegue de MLflow en EC2

Se desplegó MLflow Tracking Server en AWS EC2 (puerto 5000) con backend store SQLite y almacenamiento de artefactos en `/home/ubuntu/mlruns`. Se documentan evidencias con IP pública y usuario.

evidence/aws/EVID_AWS_01_ec2_whoami_ip.png

Figure 2: Evidencia de ejecución en EC2 (usuario e IP).

evidence/mlflow/EVID_MLFLOW_01_ui_ip.png

(a) UI MLflow con IP pública.

evidence/mlflow/EVID_MLFLOW_02_runs_list.png

Comparación de runs

evidence/mlflow/EVID_MLFLOW_03_compare.png

Figure 4: Comparación de runs en MLflow (Compare). Recomendación: mostrar `pr_auc_nochurn_0` y `f1_nochurn_0`.

Run ganador

evidence/mlflow/EVID_MLFLOW_04_best_run_detail_params_metrics.png

Figure 5: Detalle del run ganador: params y metrics.

evidence/mlflow/EVID_MLFLOW_04_best_run_detail_artifacts.png

Figure 6: Detalle del run ganador: artifacts.

6 Tablero

Descripción

Se desarrolló un tablero funcional (Streamlit) que permite:

- Seleccionar cliente (`customer_unique_id`) y filtrar por estado.
- Visualizar $p(churn)$, umbral operativo y nivel de riesgo.
- Consultar features del cliente y exportar el resultado en JSON.
- Revisar visualizaciones macro: distribución de churn y tasa de churn por estado.

evidence/dashboard/EVID_DASH_01_prediction.png

(a) Tablero: predicción individual con umbral.

evidence/dashboard/EVID_DASH_03_features_export.pn

(b) Tablero: features del cliente y export.

Figure 7: Tablero: features del cliente y export.

evidence/dashboard/EVID_DASH_02_macro.png

Figure 8: Tablero: visualizaciones macro.

7 Conclusiones y trabajo futuro

- El dataset presenta fuerte desbalance; accuracy no es suficiente. El baseline que predice churn para todos logra alta accuracy pero no identifica recompra (no-churn).

- La Regresión Logística supera a Random Forest en métricas de no-churn (PR-AUC/F1), por lo que se selecciona como modelo final.
- Se optimizó el umbral de decisión y se definió un umbral operativo de 0.12 para mejorar la detección de no-churn.
- MLflow en EC2 permitió comparar runs y conservar artefactos (matrices, curvas PR y modelos).
- Trabajo futuro: incorporar features por categoría/producto, calibración de probabilidades y evaluación basada en costos.
- La Regresión Logística superó a Random Forest en métricas de no-churn (PR-AUC/F1). El run `logreg_C0.1_thr012` se seleccionó como ganador en MLflow por su mejor `pr_auc_nochurn_0`.